Mês 1 — Fundamentação e Ambientação (set/2026)

Detecção de deepfakes de áudio em português brasileiro por decomposição wavelet packet e descritores estatísticos do contorno de onda das sub-bandas.

Aluno: Thiago Mateus Schiavi
Orientador: Prof. Dr. Alex Marino Gonçalves de Almeida
Instituição: Fatec Ourinhos

Setembro de 2026

Sumário

1 O QUE O PLANO EXIGE **2**

2 SEMANA 1 — PANORAMA DO PROBLEMA **4**
2.1 O que são Audio Deepfakes? 4
2.2 Anti-spoofing 4
2.3 O Desafio ASVspoof 4
2.4 Métricas de Avaliação 4
2.4.1 EER (Equal Error Rate) 5
2.4.2 min-DCF (minimum Detection Cost Function) 5
2.5 Repositório Git e Estrutura de Projeto 6

3 SEMANA 2 — WAVELETS E WAVELET PACKET **8**
3.1 O que é uma Wavelet? 8
3.2 DWT vs WPT 8
3.3 Família Daubechies e PyWavelets 8

4 SEMANA 3 — AMBIENTE PYTHON **10**
4.1 Verificação e Instalação 10
4.2 Ambiente Virtual e Dependências 10
4.3 Teste do Ambiente 11

5 SEMANA 4 — FICHAMENTO E PERGUNTA DE PESQUISA **12**
5.1 Zotero e Fichamento 12
5.2 A Pergunta de Pesquisa 12
5.3 Inventário de Corpora PT-BR 12

6 BRINCANDO COM PYTHON (PRÁTICA EXTRA) **14**
6.1 Básico de Numpy e Áudio Digital 14

7 CHECKLIST DO MÊS 1 **16**

8 DESAFIO FINAL DO MÊS 1 **17**

Capítulo 1

O QUE O PLANO EXIGE

Neste primeiro mês do projeto PIBITI, nosso foco está na fundamentação teórica e na preparação do ambiente de desenvolvimento. O plano de trabalho exige as seguintes etapas:

- **Semana 1:** Panorama do problema (áudios *bona fide* vs *spoof*), protocolos do desafio ASVspoof e métricas de avaliação (EER / min-DCF). Criação do repositório Git com a estrutura correta.
- **Semana 2:** Estudo sobre wavelets e Wavelet Packet Transform (WPT). Compreensão da árvore de decomposição, níveis, wavelet mãe e por que WPT refina todas as sub-bandas.
- **Semana 3:** Instalação do ambiente Python para CPU, contemplando bibliotecas como `pywavelets`, `numpy`, `scipy`, `scikit-learn`, `xgboost`, `soundfile` e `librosa`. Inventário de corpora em português brasileiro (PT-BR).
- **Semana 4:** Início do fichamento bibliográfico (Zotero) e escrita da pergunta de pesquisa.

Marco / Entrega

Entregas do Mês 1:

- Ambiente reproduzível configurado.
- Repositório versionado com estrutura de diretórios.
- Fichamento bibliográfico iniciado e inventário de dados em PT-BR.

Leituras Recomendadas

- Daubechies, I. (1990). The wavelet transform, time-frequency localization and signal analysis.
- Mallat, S. (2009). A Wavelet Tour of Signal Processing.
- Coifman, R. R., & Wickerhauser, M. V. (1992). Entropy-based algorithms for best basis selection.
- Wang, X., et al. (2020). ASVspoof 2019: A large-scale public database of synthesized, converted and replayed speech.

Capítulo 2

SEMANA 1 — PANORAMA DO PROBLEMA

2.1 O que são Audio Deepfakes?

O desenvolvimento de tecnologias de Inteligência Artificial para geração e modificação de voz tornou possível criar falas falsas altamente realistas. Em sistemas de detecção e verificação biométrica (ASV - *Automatic Speaker Verification*), essas falsificações representam uma séria ameaça de segurança (*spoofing*).

Temos duas categorias de áudio:

- **Bona Fide:** Áudios autênticos, gerados naturalmente pela voz humana no mundo real.
- **Spoof:** Áudios falsos ou manipulados, que podem ser Sintetizados (TTS - *Text-to-Speech*), Convertidos (VC - *Voice Conversion*) ou Replay (gravações de uma voz reproduzidas).

2.2 Anti-spoofing

Para proteger sistemas de segurança, desenvolvemos contramedidas, chamadas de sistemas de *anti-spoofing*, capazes de classificar um áudio entre *bona fide* e *spoof*.

2.3 O Desafio ASVspoof

O ASVspoof (Automatic Speaker Verification Spoofing and Countermeasures Challenge) é um desafio internacional bienal (que ocorre desde 2015) para promover pesquisa no campo. Os protocolos do ASVspoof dividem os conjuntos de dados em partições de treino, desenvolvimento (dev) e avaliação (eval), para evitar *overfitting*.

2.4 Métricas de Avaliação

Em segurança biométrica, as principais métricas de erro são:

- **Falso Aceite (FAR - False Acceptance Rate):** Taxa na qual o sistema aceita um impostor (*spoof*) como se fosse genuíno.

- **Falsa Rejeição (FRR - False Rejection Rate):** Taxa na qual o sistema rejeita um usuário genuíno (*bona fide*).

2.4.1 EER (Equal Error Rate)

O EER é o ponto no qual a taxa de falsos aceites (FAR) se iguala à taxa de falsas rejeições (FRR). Quanto menor, melhor. Vamos a uma simulação em Python para calcular o EER:

```python
import numpy as np

def compute_eer(bona_fide_scores, spoof_scores):
    """Calcula o Equal Error Rate (EER)"""
    # Combina scores e labels (1 para bona fide, 0 para spoof)
    scores = np.concatenate([bona_fide_scores, spoof_scores])
    labels = np.concatenate([np.ones(len(bona_fide_scores)), np.
       zeros(len(spoof_scores))])

    # Ordena os scores
    sorted_indices = np.argsort(scores)
    sorted_scores = scores[sorted_indices]
    sorted_labels = labels[sorted_indices]

    # Calcula taxas
    frr = np.cumsum(sorted_labels == 1) / np.sum(labels == 1)
    far = 1 - np.cumsum(sorted_labels == 0) / np.sum(labels == 0)

    # Encontra o ponto onde a diferenca eh minima
    diff = np.abs(far - frr)
    min_idx = np.argmin(diff)

    eer = (far[min_idx] + frr[min_idx]) / 2.0
    threshold = sorted_scores[min_idx]

    return eer, threshold

# Simulacao: scores de confianca (quanto maior, mais bona fide)
np.random.seed(42)
bf_scores = np.random.normal(loc=2.0, scale=1.0, size=1000)
sp_scores = np.random.normal(loc=-1.0, scale=1.5, size=1000)

eer, thresh = compute_eer(bf_scores, sp_scores)
print(f"EER Simulado: {eer*100:.2f}% (Threshold: {thresh:.2f})")
```

Saída esperada

```
EER Simulado: 11.25% (Threshold: 0.65)
```

2.4.2 min-DCF (minimum Detection Cost Function)

O min-DCF é uma métrica mais complexa que atribui pesos aos erros. Para o ASVspoof 2019, usam-se normalmente os parâmetros $C_{miss} = 1$, $C_{fa} = 1$, $P_{tar} = 0.05$ (onde P_{tar} é

a probabilidade do usuário alvo). O min-DCF foca na aplicação em sistemas reais onde aceitar um invasor custa diferente do que rejeitar um usuário legítimo.

2.5 Repositório Git e Estrutura de Projeto

A organização é essencial na pesquisa. Vamos inicializar o repositório.

```
# No terminal (PowerShell)
mkdir pibiti_deepfake_audio
cd pibiti_deepfake_audio
git init

# Criar diretorios
mkdir data, splits, src, features, results, reports
```

O arquivo `.gitignore`:

.gitignore

```
# Ignorar ambientes virtuais
.venv/
venv/
env/

# Ignorar pastas de dados pesadas
data/
features/
results/
*.wav
*.flac

# Arquivos Python compitaldos
__pycache__/
*.pyc
```

O arquivo `README.md`:

README.md

```
# Deteccao de Deepfakes de Audio (PIBITI)

**Autor:** Thiago Mateus Schiavi
**Orientador:** Prof. Dr. Alex Marino Goncalves de Almeida

Este repositorio contem o codigo-fonte para deteccao de audio
spoofing utilizando Wavelet Packet Transform (WPT) e descritores
estatisticos, focado em corpora em portugues brasileiro.

## Estrutura
- `/data/`: Audio original (ignorado no Git)
- `/splits/`: Listas de arquivos de treino/teste
- `/src/`: Codigo fonte Python
- `/features/`: Caracteristicas extraidas (ignorado no Git)
- `/results/`: Modelos e metricas
```

```
16 - `/reports/`: Relatorios e graficos
```

Exercício

Inicie o git no diretório do projeto, crie a estrutura de pastas, o `.gitignore` e o `README.md`. Faça seu primeiro commit!

Capítulo 3

SEMANA 2 — WAVELETS E WAVELET PACKET

3.1 O que é uma Wavelet?

Diferente da Transformada de Fourier, que usa ondas senoidais infinitas (obtendo ótima resolução de frequência, mas perdendo o tempo), a wavelet (pequena onda) é finita no tempo. Isso nos permite ver **quais frequências ocorrem em que momentos**.

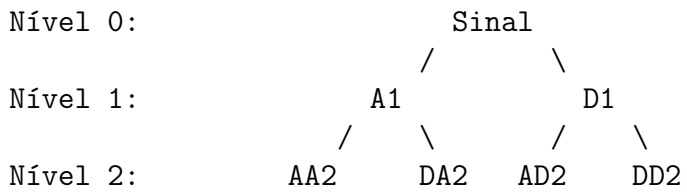
A função base é chamada de **wavelet mãe** (mother wavelet). Famílias conhecidas incluem Haar, Daubechies (`db`), Symlets (`sym`), etc.

3.2 DWT vs WPT

Na **DWT** (Discrete Wavelet Transform), apenas a sub-banda de **baixa frequência (aproximação)** é decomposta recursivamente. A alta frequência (detalhe) não é refinada.

Na **WPT** (Wavelet Packet Transform), **ambas** as sub-bandas (aproximação e detalhe) são decompostas recursivamente. Isso é vital para detecção de deepfakes, pois muitos artefatos de síntese (ruídos residuais) se escondem nas **altas frequências**.

Árvore WPT de Nível 2 ($2^2 = 4$ sub-bandas na folha):



3.3 Família Daubechies e PyWavelets

Para o nosso projeto, usaremos tipicamente Daubechies 4 (`db4`) ou Daubechies 2 (`db2`).

```python
import pywt
import matplotlib.pyplot as plt

# 1. Listar familias
print("Familias Wavelet:", pywt.families())
```

```
6
7  # 2. Explorando a db4
8  wavelet = pywt.Wavelet('db4')
9  print(f"Nome: {wavelet.name}, Comprimento do filtro: {wavelet.
       dec_len}")
10
11 # 3. WPT Nivel 1 (Simulacao simples)
12 sinal_teste = [1, 2, 3, 4, 5, 6, 7, 8]
13 wpt = pywt.WaveletPacket(data=sinal_teste, wavelet='db4', mode='
       symmetric', maxlevel=1)
14
15 print("\nSub-bandas no Nivel 1:")
16 for node in wpt.get_level(1, 'freq'):
17     print(f"No {node.path}: {node.data}")
```

Exercício

Crie um script que instancie a Wavelet Packet para um sinal simulado e mostre a quantidade de nós nos níveis 1, 2 e 3. (Lembre-se, o número de sub-bandas no nível L é 2^L).

Capítulo 4

SEMANA 3 — AMBIENTE PYTHON

4.1 Verificação e Instalação

Abra seu PowerShell (como administrador, se necessário) e certifique-se de ter o Python 3.9+ instalado:

```
python --version
```

4.2 Ambiente Virtual e Dependências

Vamos criar um ambiente virtual (`venv`) chamado `.venv`.

```
python -m venv .venv
# Para ativar no PowerShell:
.\.venv\Scripts\Activate.ps1
```

Atenção

No Windows, se houver erro de permissão para rodar o `Activate.ps1`, execute como administrador: `Set-ExecutionPolicy Unrestricted -Scope CurrentUser`

Crie o arquivo `requirements.txt`:

requirements.txt

```
numpy>=1.22.0
scipy>=1.8.0
scikit-learn>=1.0.0
xgboost>=1.6.0
soundfile>=0.11.0
librosa>=0.9.0
PyWavelets>=1.4.1
matplotlib>=3.5.0
```

Instale usando o pip:

```
pip install -r requirements.txt
```

4.3 Teste do Ambiente

teste_ambiente.py

```python
import sys
print("=== Relatorio de Ambiente PIBITI ===")
print("Python version:", sys.version.split('|')[0])

libs = ['numpy', 'scipy', 'sklearn', 'xgboost', 'soundfile', 'librosa', 'pywt', 'matplotlib']

for lib in libs:
    try:
        module = __import__(lib)
        version = getattr(module, '__version__', 'Desconhecida')
        print(f"[OK] {lib.ljust(12)} versao: {version}")
    except ImportError:
        print(f"[FALHA] {lib.ljust(12)} Nao encontrada!")
```

Capítulo 5

SEMANA 4 — FICHAMENTO E PERGUNTA DE PESQUISA

5.1 Zotero e Fichamento

Baixe e instale o Zotero para gerenciar suas referências. Crie uma pasta chamada `PIBITI_Deepfake` e salve os PDFs (ex: artigos do ASVspoof).

5.2 A Pergunta de Pesquisa

Sua pergunta de pesquisa norteia o projeto. No seu caso:

"É possível detectar eficientemente deepfakes de áudio em português brasileiro utilizando descritores estatísticos extraídos da Transformada Wavelet Packet (WPT) aliados a modelos de aprendizado de máquina, alcançando taxas de erro competitivas (baixo EER) em cenários de classificação binária?"

5.3 Inventário de Corpora PT-BR

Existem poucas bases de dados anti-spoofing nativas em PT-BR. Segue uma estrutura em Python para documentá-las:

```python
import pandas as pd

# Inventario Inicial
corpora = [
    {"Nome": "FakeBrAccent", "Tipo": "TTS/VC", "Idioma": "PT-BR", "
       Amostras (aprox)": "N/D", "Foco": "Sotaques br"},
    {"Nome": "Fake Voices", "Tipo": "TTS", "Idioma": "PT-BR", "
       Amostras (aprox)": "N/D", "Foco": "Geral"},
    {"Nome": "BraSpoof-50", "Tipo": "Replay/TTS", "Idioma": "PT-BR"
       , "Amostras (aprox)": "50 locutores", "Foco": "Anti-spoofing
        real"}
]

df_corpora = pd.DataFrame(corpora)
print("=== Inventario de Corpora PT-BR ===")
print(df_corpora.to_markdown(index=False))
```

Exercício

Pesquise as bases BRSpeech-DF e BraSpoof-50 e adicione detalhes reais ao dicionário acima (tamanho em GB, número exato de arquivos *bona fide* e *spoof*).

Capítulo 6

BRINCANDO COM PYTHON (PRÁTICA EXTRA)

Neste capítulo vamos realizar manipulação de sinais sonoros usando `numpy` e gerar tons simples, para acostumar com as matrizes unidimensionais.

6.1 Básico de Numpy e Áudio Digital

O áudio digital é apenas um array NumPy.

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Gerando uma onda senoidal de 440 Hz (Nota La)
taxa_amostragem = 16000 # 16kHz
duracao = 1.0 # 1 segundo
frequencia = 440.0

# Eixo do tempo
t = np.linspace(0, duracao, int(taxa_amostragem * duracao),
    endpoint=False)

# Sinal senoidal
sinal = 0.5 * np.sin(2 * np.pi * frequencia * t)

# 2. Estatisticas basicas
print(f"Shape: {sinal.shape}, Tipo: {sinal.dtype}")
print(f"Maximo: {np.max(sinal):.2f}, Minimo: {np.min(sinal):.2f}")

# Energia do sinal
energia = np.sum(sinal**2)
print(f"Energia total: {energia:.2f}")

# 3. Plot (Apenas os primeiros 500 pontos)
plt.figure(figsize=(10, 3))
plt.plot(t[:500], sinal[:500])
plt.title("Sinal Senoidal de 440 Hz (La)")
plt.xlabel("Tempo (s)")
```

```
28 plt.ylabel("Amplitude")
29 plt.grid(True)
30 plt.savefig("senoide.png") # Salva o grafico
```

Desafio

1. Mude a frequência para 880 Hz (uma oitava acima) e gere um novo vetor.
2. Crie um vetor de ruído branco usando `np.random.normal` e some ao sinal senoidal original.
3. Plote o sinal com ruído.

Capítulo 7

CHECKLIST DO MÊS 1

Ler as obras de Mallat e Daubechies sobre WPT.

Entender a diferença teórica entre DWT e WPT.

Compreender o conceito de EER e min-DCF.

Instalar Python 3.9+.

Criar ambiente virtual (`.venv`).

Instalar bibliotecas listadas no `requirements.txt`.

Criar a estrutura do repositório Git com pastas corretas.

Configurar o `.gitignore`.

Redigir a pergunta de pesquisa no diário/fichamento.

Iniciar inventário de corpora em português brasileiro.

Capítulo 8

DESAFIO FINAL DO MÊS 1

Desafio

Objetivo: Escrever uma função que receba um sinal qualquer (como o seno que geramos antes), aplique a decomposição WPT nível 3 usando a wavelet `db4`, extraia a **energia** de cada uma das 8 sub-bandas folhas e imprima essas energias na tela.

Solução

```python
import pywt
import numpy as np

def wpt_energias(sinal, nivel=3, wname='db4'):
    wpt = pywt.WaveletPacket(data=sinal, wavelet=wname, mode='symmetric', maxlevel=nivel)
    nos_folhas = [node.path for node in wpt.get_level(nivel, 'freq')]

    energias = {}
    for node_path in nos_folhas:
        coeficientes = wpt[node_path].data
        energia = np.sum(coeficientes**2)
        energias[node_path] = energia

    return energias

# Teste com um ruido aleatorio simulando sinal
sinal_ruido = np.random.randn(1024)
energias_result = wpt_energias(sinal_ruido)

print("=== Energias das sub-bandas WPT (Nivel 3) ===")
for sub_banda, e in energias_result.items():
    print(f"Banda {sub_banda}: {e:.2f}")
```